Security vs. Speed:

Evaluating AI in Time-Constrained Encryption

Project Paper

Group 1 Members:

Margo Bonal

Luke Ruffing

Evan Thompson

Professor: Dr. Nader Mohamed

Course: CMSC-4200-AI

School: Pennsylvania Western University

Table of Contents

Project Title: 3
Introduction 3
Description 3
Problem Types: 4
Approach 4
AI Tools 4
Analysis 5
Model 1: ChatGPT 5
Model 2: Gemini 9
Model 3: Copilot 13
Investigation/Observations 19
Limitations 19
Success Rate 19
Recommendations 19
Future of AI/Impact 20
Conclusion 20
References 21

Project Title:

Security vs. Speed: *Evaluating AI in Time-Constrained Encryption*

Introduction

AI and Cybersecurity are two fundamental fields in computer science. AI focuses on building systems that can automate performing tasks that require human intelligence. AI implements models to learn, reason, build perception, and understanding to approach problems with strategy. The article, ***What is artificial intelligence (AI)***, tells us of this innovation. *"Artificial intelligence is a branch of computer science that creates systems and software capable of tasks once thought to be uniquely human. It enables machines to learn from experience, adapt to new information, and uses data, algorithms and computational power to interpret complex situations and make decisions with minimal human input. AI can understand language, recognize patterns, solve problems and even demonstrate creativity – often at speeds and scales far beyond our own. By harnessing these capabilities, it is reshaping how we interact with technology and pushing the boundaries of what machines can achieve (2026, ISO)."*

This advancement enables a new scope of problem formulation outside of designing software for explicit and specific scenarios. AI introduces this flexibility into various fields and industries. Similarly, Cybersecurity is a crucial field for computing, since it represents guidelines and protocols of data storage, protection, and processing. Cybersecurity is both a profession and a mindset. It requires skilled engineers to develop secure solutions and architectures. However, the question remains, how does AI integrate into the field of Cybersecurity? Is AI capable of creating and abiding by Cybersecurity principles of the CIA Triad: *Confidentiality, Integrity, Availability*? Additionally, do we think that AI can provide stand-alone solutions without human intervention? Our AI class team is not only passionate about data encryption and cryptography but also captivated by answering these artificial intelligence questions. Only through careful research can we find a conclusion on whether AI can effectively solve data encryption problems in the field of Computer Science or not.

Description

Since both AI and Cybersecurity are both vast fields. For this project, scope has been limited to a specific topic in computing, Encryption. In this AI project, we would like to see if AI is capable of providing and designing encryption solutions when given a computational time constraint. For example, can AI provide optimal data hashing solutions in terms of Big O? Can AI make decisions on security level vs computational cost? Encryption is much more than just formatting data into unreadable and unrecognizable content. Encryption must meet the requirements of the scenario it is used in. These requirements are constraints of time, memory, and computing power. Is an algorithm considered to be correct if it runs several minutes over when it was proposed to give a solution? When designing encryption methods, consideration is spent on catering the proposed solution in a way that meets the problem's security and efficiency needs. This careful software design is usually done by a qualified engineer whose goal is to perfect this optimization trade off. Herein lies our project topic. Our team took great interest in whether AI could replace the software engineer's position of Encryption evaluation based on computational time complexity. This topic of evaluating AI's ability to problem solve encryption solutions when held to time constraints explores both mathematical and algorithm processing

while applying to an important computer science topic. This project has been designed to push generative AI models in both capability and comprehension.

Problem Types:

The following research paper will cover four main categories of investigation. Algorithm selection, Optimization, Complexity, and Security Analysis will represent the categories AI will be tested against. Algorithm Selection will test AI's ability and knowledge of encryption algorithms. Optimization will test the ability to select an algorithm for a specific purpose. The categories of Complexity and Security will examine AI's ability to create an encryption solution that meets a provided time complexity while ensuring data security. Each category was selected to show contrasting information highlighted to provide the best overall thesis at the end of AI model experimentation. During analysis, both basic and complex problems will be selected to ensure a range of data to gather. These 4 main problem types to investigate will provide an overall conclusion of AI's ability in data encryption. During these experiments, limitations will be tracked and discussed further in the report.

Approach

Each team member has been assigned an AI model for experimentation. The AI model will be prompted with specific queries to generate encryption algorithms. Generated content will then be tested for optimization and complexity. Additionally, a final security analysis will be given on each generated encryption algorithm. For smooth processing and cohesive project analysis, our team will stay in consistent communication during experimentation and provide peer reviews of team solutions. Each member of the team will additionally provide report documentation under specified categories to summarize findings. This contribution will allow for a final conclusion of AI's encryption and time constraints ability.

AI Tools

The following tabular representation demonstrates the generative AI models that we have selected to perform this research project.

Agent	Foundation
ChatGPT	GPT
Copilot	GPT (with additional Microsoft layers and modifications)
Gemini	Gemini

Utilizing multiple AI agents will allow for comparison of success rates, solutions, and limitations. ChatGPT and Copilot both exist on the foundation of OpenAI's GPT (Generative Pre-trained Transformer) models, while Gemini exists via Google's own independent models. Although they share the same foundations, ChatGPT and Copilot are not simply copies of one another. Microsoft utilizes GPT models but adds layers to allow Copilot to meet their security needs and to integrate closely with the broader Microsoft ecosystem.

GPT-based models, developed by OpenAI, have overwhelming market share among large language models (Statcounter Global Stats, 2026). Given this, our selection of AI agents gives a variety of options: GPT-based, modified GPT-based, and non GPT-based. In comparing results, our team may be able to determine if one agent or foundation type is more successful or efficient for our application than the others.

Analysis

Model 1: ChatGPT

For the first AI model experimentation, ChatGPT will be prompted by queries from the four different categories in attempt to move towards an answer based on the findings. Each prompt will be performed on a new chat to prevent previous memory and history from interfering with the validity or persuasion of the data. Each prompt will be designed with enough information for the model to solve, however, it will be checked for any inflection that would convince the model of a specific solution prior to processing.

Algorithm Selection

This category seeks to answer the question of whether AI can properly choose an encryption algorithm while keeping in mind speed, efficiency, and security.

<u>Prompt</u>:

A datacenter needs to encrypt their employee's system passwords. The passwords are required to be 8 characters including numbers and special characters. The software needs to encrypt the password upon account creation and verified on relogging in. There needs to be no lag during this process. There are 200 employees in this datacenter that all need accounts. Which encryption approach should be used and why?

ChatGPT broke prompt 2 into the several components below represented in the table.

Constraints	Approach	Architecture
No Lag Requirement • Hashing a single password takes milliseconds • Login verification is extremely fast 2. High Security • Slow down brute force attacks • Use salt to prevent rainbow table attacks 3. Small Data Size Passwords are 8 characters, so hashing is extremely fast and efficient.	Encryption can be reversed with a key = dangerous Password systems one-way hashing: 1. User creates password 2. System generates a random salt, hashed 3. Only the hash and salt are stored User logs in: 1. The entered password is hashed again with the stored salt 2. Compare hash to store 3. match → success	Creation: password -> add salt -> hash with Argon2 -> store hash + salt Verification: entered password -> retrieve salt -> hash again -> compare hashes The actual password is never stored.

ChatGPT Answer Conclusion:

The system should use secure salted password hashing with an algorithm such as Argon2 or bcrypt rather than encryption. These algorithms are designed specifically for password storage, provide strong protection against brute force attacks, and are fast enough that hashing and verification for 200 employees will occur with virtually no noticeable latency during account creation or login. (GPT, 2026)

Optimization

This category seeks to answer the question of whether AI can properly optimize an encryption algorithm. AI must be able to take into account specified requirements as an engineer would to determine what tradeoffs must be made, while maintaining compliancy. The following prompt has been designed to represent both selection features and optimization strategies.

Prompt:

A system must encrypt 5GB of data per hour on limited hardware. High security is required, but latency must remain under 200ms per transaction. Which encryption approach should be used and how can it be optimized?

ChatGPT broke prompt 1 into the several components below represented in the table.

Constraints	Approach	Architecture
• 5 G/hr. throughput • Limited hardware • High security • Latency < 200 ms per transaction Best approach is hybrid encryption, symmetric cipher (AES) with asymmetric key exchange (RSA or ECC).	Symmetric encryption (AES) for data & asymmetric (ECC/RSA) key exchange.	• Client generates random AES session key • AES key exchanged using ECDH • Data encrypted using AES-256-GCM • Encrypted data transmitted

Process
1. Symmetric Encryption for Data i. AES-256 GCM provides authentication & encryption ii. Fast compared to asymmetric iii. Allows large data handling iv. Hardware accelerated v. Easily encrypt hundreds of MB/s on smaller CPUs, Meets 5GB/hr.
2. Asymmetric Encryption for Key Exchange a. Secure exchange of symmetric session key b. Smaller keys allow faster operation

Figure 1: Requirements Table. (Source: OpenAI- ChatGPT)

Requirement	How Hybrid Encryption Solves It
High security	AES-256 + strong key exchange
5GB/hour throughput	AES handles large data quickly
Limited hardware	Symmetric encryption is lightweight
<200ms latency	Only key exchange uses slower crypto

ChatGPT Answer Conclusion:

The use of hybrid encryption: AES-256-GCM for bulk data encryption with ECC for secure key exchange provides optimization, because symmetric encryption provides the necessary throughput and low latency. Asymmetric cryptography ensures secure key distribution. The combination of asymmetric and symmetric encryption reduces slow and CPU intensive processing. Single asymmetric encryption would also exceed the latency since it is not feasible for large data streams or payloads. Optimization can be further achieved through session key reuse within secure time windows. Additionally, efficient memory buffering of large data streams could be implemented. These optimizations reduce the frequency of expensive cryptographic operations while maintaining strong security guarantees. As a result, the system can reliably encrypt large volumes of data while operating within strict hardware and latency constraints. (GPT, 2026)

Complexity

This section represents the most crucial research point of our project, since focused on specific time complexity. This section will test AI's ability to produce solutions that meet a certain computational time requirement.

<u>Prompt</u>: Design a small encryption algorithm to encrypt license numbers in constant time then in log time in python.

Constant time	Log Time
Process: 1. Convert characters → ASCII 2. XOR with a secret key 3. Rotate bits 4. Convert back to characters	Process: • Keys stored in a balanced binary tree • Tree lookup = O (log n) K50 / \ K25 K75 / \ / \ K10 K30 K60 K90

ChatGPT Answer Conclusion:

Constant-Time encryption:

By implementing a fixed key, apply XOR, and bit rotations to each character of the license number, constant time encryption can be achieved. Since licenses numbers have fixed lengths, the number of operations remains constant, giving an **O (1)** complexity.

Logarithmic-Time encryption:

However to specifically achieve Log Time, encryption keys need to be stored by a balanced binary search tree. For further processing, they must be selected with a key based on a hash of the license number. Tree lookups require **O (log n)** time; therefore, the encryption process is logarithmic.

The figure below demonstrates the code generated from ChatGPT AI model. Code will be included upon submission.

Security Analysis

<u>Prompt</u>: Provide a solution to hash a bank's transaction history, sacrifice computational time to prove the best security.

Goal: maximum security for a bank's transaction history, sacrifice computational time, Process: use a slow, memory-hard hashing strategy with chaining - Collision resistance → use a strong hash (e.g., SHA-256 or SHA-3) - Slow computation → use a memory-hard KDF like Argon2 - Tamper detection → chain hashes like a blockchain

Figure 2: Security vs Time. (Source: OpenAI- ChatGPT)

Approach	Time	Security
Simple SHA256	Fast	Medium
Iterated SHA256	Slow	High
Argon2 Memory Hard	Very Slow	Very High
Chain + Argon2	Extremely Slow	Maximum

The figure below demonstrates the code generated from ChatGPT AI model. Code will be included upon submission.

```
----Entry {'transaction': 'TX001: Alice -> Bob $500', 'previous_hash': 'GENESIS', 'hash': '71fdfd3207bd0f43216634216ccc9ff6e6f1df8
e727e61ccd8d42c30880fd16f'} ---

----Entry {'transaction': 'TX002: Bob -> Carol $200', 'previous_hash': '71fdfd3207bd0f43216634216ccc9ff6e6f1df8e727e61ccd8d42c3088
0fd16f', 'hash': 'cefb8248578084e7729c850d6f3d334bc4199052165dc31e6748baa08eca1069'} ---

----Entry {'transaction': 'TX003: Carol -> Dave $50', 'previous_hash': 'cefb8248578084e7729c850d6f3d334bc4199052165dc31e6748baa08e
ca1069', 'hash': '2cd39a8f64ef2c4be14cfa141e5ea4cda9c10f0b84d29ca22df0a792a81a5783'} ---

----Entry {'transaction': 'TX004: Dave -> Bank $10', 'previous_hash': '2cd39a8f64ef2c4be14cfa141e5ea4cda9c10f0b84d29ca22df0a792a81
a5783', 'hash': '17addf75a9da346bea32f7d81fb68c5a26af6d901976b1053e1cdf80f0b11f3c'} ---


Time taken: 1.4470758438110352 seconds
margo  CMSC-4200-A1  →(main)  ▼ 11:24
```

Model Summary

Chat GPT AI model was able to take the proposed questions and formulate solutions. Breaking the prompts into sections allowed for thorough and logical processing. Each section prompt allowed for diverse testing information. During this ChatGPT experimentation, our team was able to see both pros and cons. The AI model has great capabilities of processing the prompt. It can separate specific requirements from the prompt seamlessly. By formatting the requirements into tabular representations, the AI model reached optimal problem organization. Solutions were provided quickly with through explanations. However, it was noticed that ChatGPT provided no references or reasons behind these solutions. The user or engineer is required to blindly trust AI or do external research that would add additional time to project composition. This produces a detrimental cost when dealing specifically with code since it is the intellectual property of the author. However, without proper documentation, who is the author of the AI generated code or logic? Has it been sourced from an external internet source? Has it been reused or modified? Overall credibility is a major concern regarding this AI model.

Model 2: Gemini

For the second AI model experimentation, Google Gemini will be prompted with the exact same prompts that were sent to ChatGPT. Each prompt will be performed on a new chat to prevent previous memory and history from interfering with the validity or persuasion of the data. Gemini is unique from the other two models explored in this paper, in that it has its own unique foundational LLM; it does not use a GPT model, as ChatGPT and Copilot do.

Algorithm Selection

This category seeks to answer the question of whether AI can properly choose an encryption algorithm while keeping in mind speed, efficiency, and security.

<u>Prompt:</u>

A datacenter needs to encrypt their employee's system passwords. The passwords are required to be 8 characters including numbers and special characters. The software needs to encrypt the password upon account creation and verified on relogging in. There needs to be no

lag during this process. There are 200 employees in this datacenter that all need accounts. Which encryption approach should be used and why?

Gemini responded, recommending Salted Hashing with Argon2id or BCrypt. It specified that "encrypt" is not the correct action for what is needed, as encryptions can be decrypted. It presents hashing as distinct from encryption, and provided the following table:

Feature	**Symmetric Encryption (AES)**	**Standard Hashing (SHA-256)**	**Salted Hashing (Argon2/BCrypt)**
Reversibility	Reversible (Riskier)	Non-reversible	Non-reversible
Speed	Very Fast	Too Fast (Easy to crack)	Optimized (Secure but quick)
Best Use Case	Data at rest (Files)	Data integrity	Password Storage

Gemini recommended that the size of passwords be increased, citing 8 characters as the bare minimum for this use case. It also said that it is vital that the software used to hash the passwords uses a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG) for the system to work as required.

Gemini Answer Conclusion:

For a datacenter handling 200 employees, the most effective approach is Salted Hashing using Argon2id. This method is the modern gold standard because it provides robust protection against brute-force and hardware-accelerated attacks while maintaining the "zero-lag" performance required for a smooth user experience. By storing a unique, non-reversible hash rather than an encrypted password, the datacenter ensures that even if the database is compromised, the actual passwords remain unreadable. For an 8-character requirement, this approach provides the necessary security depth without requiring expensive infrastructure or causing noticeable login delays. (Gemini, 2026)

Optimization

This category seeks to answer the question of whether AI can properly optimize an encryption algorithm. AI must be able to take into account specified requirements as an engineer would to determine what tradeoffs must be made, while maintaining compliancy. The following prompt has been designed to represent both selection features and optimization strategies.

<u>Prompt:</u>

A system must encrypt 5GB of data per hour on limited hardware. High security is required, but latency must remain under 200ms per transaction. Which encryption approach should be used and how can it be optimized?

Gemini recommended the AES-GCM approach to meet the requirements. Gemini provided the following justification for its decision:

- Security: AES-GCM is virtually unbreakable by brute force

- Performance: AES-GCM is parallelizable, unlike older models like CBC, which is critical for hitting 5GB/hr
- Efficiency: AES-GCM combines encryption and authentication into one operation, reducing total overhead

Gemini further suggested breaking the 5GB data into smaller blocks of 64KB or 128KB, to keep individual transaction latency low. In addition, Gemini provided the following calculation to determine average throughput, stating the resulting value is low for a modern PC, and thus achievable using AES-GCM:

$$\text{Throughput} = \frac{5 \times 1024\ \text{MB}}{3600\ \text{seconds}} \approx 1.42\ \text{MB/s}$$

Gemini Answer Conclusion:

To achieve high-security encryption of 5GB per hour on constrained hardware while maintaining sub-200ms latency, the optimal solution is AES-GCM (Galois/Counter Mode) leveraging AES-NI or ARMv8 cryptography extensions. This approach succeeds because AES-GCM provides both encryption and integrity in a single, parallelizable pass, while hardware acceleration offloads the heavy mathematical lifting from the general CPU cycles. By breaking data into smaller 64KB chunks and utilizing asynchronous, zero-copy memory patterns, you can comfortably meet the ~1.42 MB/s throughput requirement without blocking system resources or exceeding your strict latency budget. (Gemini, 2026)

Complexity

This section represents the most crucial research point of our project, since focused on specific time complexity. This section will test AI's ability to produce solutions that meet a certain computational time requirement.

<u>Prompt:</u>

Design a small encryption algorithm to encrypt license numbers in constant time then in log time in python.

Constant time: perform a fixed number of operations regardless of input size. Use static key and bit-shuffling. Hides a number by mixing it with a secret key and rearranging its bits. Reverse those steps to recover the original value.

Log time: The number of processing steps grows logarithmically relative to the value of the input as the input grows. Repeatedly scrambles a number multiple times, proportional to the size of the number in bits. Each scramble, use a slightly different key and rotate the bits.

Feature	**Constant Time O(1)**	**Log Time O(logn)**
Speed	Extremely fast; fixed operations.	Slower; scales with input size.
Security	Very weak (easily reversible).	Slightly better due to multiple rounds.

Feature	Constant Time O(1)	Log Time O(logn)
Use Case	Simple obfuscation.	Variable-length ID masking.

The figure below demonstrates the output of the code generated by Google Gemini. Code will be included upon submission.

```
evan@evan-ubuntu:~/Documents/CMSC-4200-AI/Project/paper/code$ python3 Gemini-complexity.py
--- Testing License: ABC1234 (Int: 180097588) ---

[Constant Time O(1)]
Encrypted: 386537427
Decrypted: ABC1234

[Logarithmic Time O(log n)]
Encrypted: 154697220
Decrypted: ABC1234
evan@evan-ubuntu:~/Documents/CMSC-4200-AI/Project/paper/code$ |
```

Security Analysis

Prompt: Provide a solution to hash a bank's transaction history, sacrifice computational time to prove the best security.

Gemini presented the goal as not just hiding data, but making it computationally expensive to try to crack it (key stretching). To achieve this, Memory-Hard Functions should be used, which resist specialized hardware such as ASICs or GPUs, by requiring a large amount of RAM to compute. Gemini recommended the algorithm Argon2id, and using a Hash Tree structure, not hashing the whole file as one block. Parameters can be set such that a single hash takes ~0.5 to 1 second to complete.

The following table was given by Gemini as an overview of each component of the system:

Parameter	Recommended Value	Purpose
Algorithm	Argon2id	Hybrid approach for maximum resistance.
Memory Cost	2GB - 4GB	Forces attackers to use massive amounts of expensive RAM.
Time Cost	10+ Iterations	Increases the time required for each guess.
Salt	16+ Bytes	Unique per transaction to prevent Rainbow Table attacks.

The architecture around the function was presented by Gemini as the real source of security, beyond the function used. Gemini is quoted below:

"**Peppering:** In addition to a salt, use a "pepper"—a secret key stored in a Hardware Security Module (HSM) or a separate vault. Even if the database is leaked, the hashes cannot be computed without this secret.

Chaining: Each transaction hash should include the hash of the *previous* transaction. This creates a tamper-evident ledger where changing one record invalidates every subsequent record.

Quantum Resistance: While Argon2 is robust, ensure your overall system uses sufficiently large output lengths (e.g., 512-bit) to remain resilient against future Grover's algorithm-based attacks"

Model Summary

Google Gemini performed well in answering all prompts. However, it did not provide a clear overview of each problem before proving the answer. Gemini would provide a strong and usable answer, but would not show any detailed steps of how it worked through the problem to get to that solution. It appeared as if Gemini preferred to, whenever possible, give the bare minimum working solution, and did not want to add any more details in its response. The AI model may be intentionally tuned to use as few tokens as possible while still providing a correct response, rather than exhaustively explaining that response. In addition, Gemini did not by itself provide a summary overview of its solution, and a second prompt was required for each problem, specifically asking for the answer to be summarized into a clear, easy to understand paragraph.

Model 3: Copilot

For the third and final AI model, we will prompt Microsoft's Copilot with the same queries from the four different categories that we used for the other two models. Each prompt will be performed in a new chat to prevent previous memory and history from interfering with the validity or persuasion of the data. Each prompt will be designed with enough information for the model to solve, however, it will be checked for any inflection that would convince the model of a specific solution prior to processing.

Algorithm Selection

This category seeks to answer the question of whether AI can properly choose an encryption algorithm while keeping in mind speed, efficiency, and security.

Prompt:

A datacenter needs to encrypt their employee's system passwords. The passwords are required to be 8 characters including numbers and special characters. The software needs to encrypt the password upon account creation and verified on relogging in. There needs to be no lag during this process. There are 200 employees in this datacenter that all need accounts. Which encryption approach should be used and why?

Copilot broke the prompt into many components represented in the table below.

Constraints	Approach	Architecture
• Passwords must be stored securely. • Must support 8-character passwords with numbers & special characters • No noticeable lag for users • System must support 200 employee accounts	• Use **Argon2id** one-way hashing (not encryption) • Apply a **unique salt** per password. • Tune hashing cost so verification occurs in ~50–200 ms	**Password Creation:** 1. User creates password. 2. System generates random salt. 3. Compute Argon2id(password + salt). 4. Store hash + salt. **Login:** 1. User enters password. 2. Retrieve stored salt. 3. Recompute Argon2id hash. 4. Compare to stored hash to authenticate.

Copilot Answer Conclusion:

Use a salted, one-way password hashing algorithm—preferably Argon2id—because it securely stores passwords without ever needing to decrypt them, protects against modern attacks, and runs fast enough to avoid any noticeable delay for your 200 users. (Copilot, 2026)

Optimization

This category seeks to answer the question of whether AI can properly optimize an encryption algorithm. AI must be able to take into account specified requirements as an engineer would to determine what tradeoffs must be made, while maintaining compliancy. The following prompt has been designed to represent both selection features and optimization strategies.

<u>Prompt:</u>

A system must encrypt 5GB of data per hour on limited hardware. High security is required, but latency must remain under 200ms per transaction. Which encryption approach should be used and how can it be optimized?

Copilot broke the prompt into many components represented in the table below.

Constraints	Approach	Architecture
• Must encrypt 5 GB/hour (~1.4 MB/s throughput requirement) • High security (modern	**Primary (with AES acceleration):** • Use AES-256-GCM (AEAD, fast, low-latency,	• Streaming / chunked encryption (4–64 KB segments) • Use hardware acceleration

strong cipher, AEAD preferred) • <200 ms latency per transaction • Limited hardware (may or may not have AES acceleration)	parallelizable) **Fallback (without AES acceleration):** • Use ChaCha20-Poly1305 (excellent on constrained CPUs) • Or AES-CTR + HMAC-SHA-256 (streaming-friendly, secure)	(AES-NI, ARM Crypto Extensions, etc.) when available • Parallel block processing for throughput • Pre-generate IVs and derive session keys via HKDF to reduce per-transaction overhead • AEAD mode ensures encryption + authentication in one step • Optional crypto worker pool to keep latency consistent

Copilot Answer Conclusion:

A high-security, low-latency system on limited hardware can reliably meet its requirements by using a fast, AEAD-capable cipher such as AES-256-GCM (or ChaCha20-Poly1305 when AES acceleration is unavailable). By combining this with streaming encryption, hardware acceleration, and lightweight key/IV management, the system easily stays within the 200 ms latency window while securely processing the required data volume. (Copilot, 2026)

Complexity

This section represents the most crucial research point of our project, since focused on specific time complexity. This section will test AI's ability to produce solutions that meet a certain computational time requirement.

Prompt: Design a small encryption algorithm to encrypt license numbers in constant time then in log time in python.

Constant time	Log Time
Process: 1. Receive license number ("ABC1234") 2. Convert to bytes 3. Compute SHA-256 hash → always 32 4. Take first 8 bytes of hash → constant 5. Convert those 8 bytes to hex string	Process: 1. Receive license number ("ABC1234") 2. Convert to base-36 integer 3. Set exponent 4. Set modulus 5. Perform modular exponentiation

6. Runtime does NOT depend on input length 7. Output is fixed-size 16-char hex → O(1) constant work	6. Runtime grows with log of exponent / modulus size 7. Output is an integer within modulus range → O(log n) log work

Copilot Answer Conclusion:

Constant-Time Encryption Conclusion:

The constant-time encryption method provides a fast, predictable, and fixed-duration operation regardless of input size. By using SHA-256 and truncating the output to a fixed 8-byte segment, the algorithm ensures consistent performance with no variation based on the length or complexity of the license number. This makes it ideal for systems where timing consistency is critical—such as embedded devices, rapid validation tasks, or scenarios where timing attacks must be avoided. However, because it is based on a one-way hash, this approach cannot be decrypted and is therefore suitable only for irreversible verification rather than recoverable encryption. (Copilot, 2026)

Log-Time Encryption Conclusion

The log-time encryption method demonstrates the use of modular exponentiation, an algorithm whose performance scales logarithmically with the size of the exponent or modulus. Although the specific implementation in this program uses fixed parameters (making it effectively constant in practice), the underlying algorithm is inherently **O(log n)** and represents how many real-world cryptographic systems operate, including RSA-style encryption. This approach allows transforming the license number into a recoverable encrypted value, enabling secure reversible operations. It is more computationally expensive than the constant-time hash, but it provides a stronger cryptographic structure and the possibility of decryption using a corresponding private key. (Copilot, 2026)

The figure below demonstrates the code generated from Copilot AI model. Code will be included upon submission.

```
----CONSTANT TIME----
Encrypted: 979ae4c32c826f26
--------------------

----LOG TIME----
Encrypted: 12500128115491945779
--------------------
```

Security Analysis

This section will show how secure the AI generated security solutions are. Since this section is completely focused on the best security the AI model will be prompted to sacrifice computational time for the best security measures.

Prompt: Provide a solution to hash a bank's transaction history, sacrifice computational time to prove the best security

Copilot broke the prompt into many components represented in the table below.

Steps	What's Happening
1. Prepare transaction data	The bank serializes each transaction and generates a unique random salt.
2. Apply Argon2id hashing	The serialized transaction + salt is hashed with Argon2id using high memory and high iteration parameters.
3. Create hash chain link	Each transaction hash is combined with the previous chain hash using SHA-3-512.
4. Build Merkle tree (periodically)	Every N transactions, all hashes in that batch are placed into a Merkle tree and the Merkle root is calculated.
5. Commit Merkle roots	The Merkle root for each batch is stored internally and linked into the chain.
6. Anchor chain externally	At the end of each day, the latest chain head or Merkle root is published to an external, independent system (e.g., regulator, HSM, or public blockchain).
7. Long-term verification	Auditors or systems can verify any transaction by tracing its hash through the chain, batch Merkle tree, and anchored commitments.

Copilot made this table to show each component and how it effects the security.

Components	How It Works / Why It Improves Security
Argon2id memory hardness	Uses large amounts of RAM and memory-hard design to slow down attackers, making GPU/ASIC brute-forcing inefficient and costly.
High Argon2 parameters	Increases computational time intentionally by raising iterations, memory usage, and processing cost per transaction hash.
Hash chain	Links each transaction hash to the previous one; any modification to a past entry breaks all

	subsequent hashes, preventing reordering or deletion.
Merkle commitments	Groups transactions into a Merkle tree and stores/publishes only the root, enabling fast verification for auditors and efficient proof of inclusion.
External anchoring	Publishes daily chain or Merkle roots to independent systems (regulators, HSMs, or public ledgers), ensuring tamper evidence even against privileged insiders.
SHA-3-512	A modern, secure hash function providing strong long-term integrity and a comfortable margin against emerging cryptographic or quantum attacks.

Copilot Answer Conclusion:

By combining Argon2id's memory-hard hashing, deliberately high hashing parameters, a cryptographically linked hash chain, Merkle-tree commitments, external anchoring, and a robust modern hash like SHA-3-512, this system maximizes security by design—intentionally trading computational speed for verifiable integrity. Any attempt to alter past transactions becomes both computationally prohibitive and cryptographically detectable, even against well-resourced adversaries or insider threats. The result is a transaction-history architecture that is tamper-evident, audit-friendly, and built to remain trustworthy far into the future, providing banks with a defensible, high-assurance method for securing their most critical financial records.

Model Summary

Overall, the Microsoft Copilot AI model performed well across all four evaluation sections. It was able to formulate logical solutions for each prompt in a timely manner. Breaking each prompt into multiple sections to help with readability and to attempt to show a thought process. Although the model did provide mostly clear concise answers there were a couple problems we ran into while prompting Copilot. The first being unfinished answers for example, after being prompted during one of the experiments Copilot built a table to show its encryption steps but did not fill the table with all the steps it previously described. Although this is not a major error it does raise questions about how fickle some AI models' memory or reasoning process really are. Other problems were smaller in comparison but did make the user experience less pleasant than if they were not encountered.

Investigation/Observations

Ai has many capabilities. Through these experiments, our team was able to see both investigations and observations. Encryption algorithms seemed to be a complex topic for AI to solve. It was required for the models to use both acquired knowledge and problem-solving skills. The several encryption and security related prompts we tested AI against were not simply yes or no questions. They pushed AI to formulate the problem, give a solution, then enhance that conclusion for optimization and efficiency.

Throughout the AI experiments, similarities were found. GPT based models gave similar recommendations and solutions, while Google Gemini took different an approach of solving strategies. All three AI models were able to provide solutions for the encryption and time complexity questions. Both math and programming knowledge were implemented by the models to find an outcome.

Limitations

Even though solutions were found, limitations can be seen in Artificial Intelligences knowledge and processing. The first limitation that was common across the models was the lack of referencing or citations. Since code is intellectual property, AI pulling code from across the internet can be seen as dangerous for legality. Additionally, the method of blindly trusting an AI model's solution without any time of reference to back up the knowledge directly highlights a limitation.

The next limitation can be seen in how a prompt is processed. AI models can easily focus or hyper fixate on a certain part of the prompt, letting other important information not take precedence. When comparing speed vs security, the models were more focused on the time constraints than ensuring safety. Finally, other limitations can be seen in AI models. These were the model timing out during a prompt and not completing an answer, lack of detail, and summarization faults. However, the greatest limitation was the lack of a skilled human engineer to double check the validity of the solutions. While AI can be a beneficial tool, this project shows the importance of pairing human logic with AI. Software engineers are trained to follow simple and specific details, while AI is looking for more broad data.

Success Rate

Overall, this project resulted in a high success rate. Each prompt was given a successful answer. ChatGPT, Copilot, and Gemini were able to investigate Algorithm selection, Optimization, Complexity, and Security Analysis problems that ranged from simple to complex. Clear opinions could be made based on the findings.

Recommendations

To further test AI's capabilities, our team would recommend testing more prompts. These could include vague prompts where the models would have to make their own conclusions, or

prompts that are fully developed and detailed. It would also be interesting to test with more computational time complexities. In this project, for scope only two were selected (O (log n0) and O (1)). An additional recommendation could be to test how long it takes the model to form a whole solution.

Future of AI/Impact

The future of AI is both exciting and concerning. As aspiring software engineers, it is important for technological advances. It means our industry is growing and maintaining importance. The addition of field diversity in this industry allows for more engineers to find the right fit of study. The 2026 article, *AI Impact by 2040,* shared insight into what other engineers and experts think the future will hold. *"Some pointed out that future AI-abetted losses and gains will be unevenly distributed across humanity. Some said the future will be "scary" and some said it will bring "joy and love." Some said it will initiate "growth and productivity", but it could result in "rampant unemployment."* While AI introduces an encroaching fear of unemployment, it also has enhanced many industries, boosting productivity. Only time will tell if AI is a positive or negative feature. However, promoting growth in the computer science industry is most important. It remains our jobs to keep AI accountable and abiding by internet protocols.

For our team, we felt like this project represented the fact that AI cannot completely replace skilled software engineers. At least for now. AI still struggles with mathematics and logic-based questions due to rudimentary development. AI is still considered a newer technology even though it dates back to the 1950s and needs thorough experimentation. With continued data training, models will build their knowledge base to tackle harder problems in the future. Validation still will remain to be a key component in not only data encryption solutions, but also other software related applications.

Conclusion

While determining if AI models were capable of solving data encryption related problems while following constraints such as time complexity, our team grew in understanding of how AI models work and their skillset. ChatGPT, Copilot, and Gemini are powerful tools when used correctly. They can provide logical problem formulations, break down information into neat table formation, and even summarize prompts. These AI models were able to build code to meet certain requirements and follow computational time complexity. Comparing how the AI models solved problems was fascinating. Some worked in steps, while others collected info to fully understand the problem. This project highlighted that fully observable models lead to intricate solutions as seen in this semester's material. AI has such a vast knowledge base of algorithms and computer science. Our team was able to highlight success rates and limitations during this project. Overall, this project was found to be successful. AI is able to build data encryption solutions. However, at the current state of development we believe that it cannot replace an engineer or mathematician. AI should be supplemented as a tool, not substitute human intelligence.

References

Cisco. (2026). What is Cybersecurity?

https://www.cisco.com/site/us/en/learn/topics/security/what-is-cybersecurity.html#tabs-35d568e0ff-item-4bd7dc8124-tab

Citizens Count. (2025, Aug 11th). Understanding AI Models, Terms, and Policy.

https://www.citizenscount.org/news/understanding-ai-models-terms-and-policy?gad_source=1&gad_campaignid=304664866&gbraid=0AAAAADt27-5neV3g2zdErUtyL9SXov1WM&gclid=Cj0KCQiA2bTNBhDjARIsAK89wlGNWLYKk0xAmYz8UB7f5KZYnUuueggjm8EyOcILbQL-LO_TfZuKfGcaAkuDEALw_wcB

Elon University. (2026). AI Impact by 2040.

https://imaginingthedigitalfuture.org/reports-and-publications/the-impact-of-artificial-intelligence-by-2040/a-selection-of-future-scenarios-how-things-might-play-out/

ISO. (2026). What is artificial intelligence (AI)?

https://www.iso.org/artificial-intelligence/what-is-ai

Statcounter Global Stats. (2026). *AI chatbot market share worldwide*.

https://gs.statcounter.com/ai-chatbot-market-share